



Politechnika Warszawska

Wydział Matematyki
i Nauk Informatycznych

Metody Sztucznej Inteligencji 2

Zastosowanie Monte-Carlo Tree Search do
implementacji sztucznej inteligencji w grze
Breakthrough

Raport końcowy

Zespół:

inż. Adam Grącikowski

inż. Piotr Iśtok

Prowadzący:

mgr inż. Bartłomiej Olechno

prof. dr hab. inż. Jacek Mańdziuk

Oświadczamy, że niniejsza praca, stanowiąca podstawę do uznania osiągnięcia efektów uczenia się z przedmiotu *Metody Sztucznej Inteligencji 2*, została wykonana samodzielnie.

Warszawa, 27 maja 2026

Spis treści

1	Instrukcja użytkownika	2
1.1	Środowisko i proces budowania	2
1.2	Opis plików wykonywalnych	2
1.2.1	Środowisko graficzne	2
1.2.2	Środowisko turniejowe	3
1.3	Format pliku wejściowego	3
1.3.1	Agent Minimax	4
1.3.2	Agent MCTS	4
1.3.3	Gracz ludzki	5
1.4	Format pliku wyjściowego	5
1.5	Interfejs graficzny	7
2	Weryfikacja hipotez badawczych	9
2.1	Hipoteza 1: Wpływ algorytmu RAVE	9
2.2	Hipoteza 2: Wpływ algorytmu Heavy Playouts	9
2.3	Hipoteza 3: Przewaga nad ewaluacją statyczną	10
2.3.1	Metodologia i przebieg eksperymentu	10
2.3.2	Analiza wyników	10
2.3.3	Wnioski końcowe	14
2.4	Hipoteza 4: Wpływ rozmiaru planszy na asymetrię szans	15
2.4.1	Metodologia i przebieg eksperymentu	15
2.4.2	Analiza wyników	15
2.4.3	Wnioski końcowe	17
3	Testy z udziałem ludzi	18
4	Podsumowanie	19

1 Instrukcja użytkownika

1.1 Środowisko i proces budowania

Warstwa obliczeniowa oraz interfejs graficzny projektu zostały w całości zrealizowane w języku Rust [1]. Do poprawnego zbudowania i uruchomienia plików wykonywalnych niezbędne jest posiadanie dedykowanych dla tego języka narzędzi (ang. *Rust toolchain*). Zalecana wersja kompilatora to 1.88.0 lub nowsza.

Aby zweryfikować poprawność instalacji środowiska, należy wywołać w wierszu poleceń polecenia sprawdzające wersje kompilatora `rustc` [2] oraz menedżera pakietów `Cargo` [3] (Rysunek 1):

```
> rustc --version
rustc 1.88.0 (6b00bc388 2025-06-23)
> cargo --version
cargo 1.88.0 (873a06493 2025-05-10)
```

Rysunek 1: Weryfikacja instalacji środowiska Rust.

Aplikację należy budować w trybie optymalizacji kompilatora (ang. *release*) korzystając z terminala w głównym katalogu projektu (Rysunek 2):

```
> cd src/compute && cargo build --release
```

Rysunek 2: Budowanie aplikacji w trybie optymalizacji kompilatora.

Po udanej kompilacji, w katalogu `src/compute/target/release` wygenerowane zostaną dwa niezależne pliki wykonywalne: `breakthrough` oraz `tournament`. Każdy z programów umożliwia obsługę argumentów linii poleceń (CLI). Wbudowany system pomocy dla każdego z algorytmów można wywołać dodając flagę `-h` lub `--help` podczas uruchamiania z terminala (np. `./as --help`).

Wszystkie programy przyjmują na wejściu ścieżkę do pliku konfiguracyjnego w formacie TOML.

1.2 Opis plików wykonywalnych

W następnych podrozdziałach przedstawiono szczegółowy opis argumentów pozycyjnych, flag oraz opcjonalnych parametrów sterujących dla każdego z plików wykonywalnych.

1.2.1 Środowisko graficzne

Plik wykonywalny `breakthrough.exe` uruchamia graficzne środowisko testowe przeznaczone do pojedynczej rozgrywki pomiędzy ludźmi lub człowiekiem i agentem (jednym z zaimplementowanych algorytmów). Pozwala na zdefiniowanie wymiarów planszy, ustawienie ziarna losowości oraz zapis szczegółowych informacji z przebiegu partii osobno dla każdego z graczy (Rysunek 3).

```
> breakthrough.exe [OPTIONS] <CONFIG>
```

Rysunek 3: Podstawowe użycie graficznego środowiska testowego.

Tabela 1: Argumenty i opcje dla programu breakthrough.

Opcja / Argument	Opis
<CONFIG>	Ścieżka do pliku konfiguracyjnego TOML. Pozycja opcjonalna, domyślnie przyjmuje wartość <code>config.toml</code> .
--white-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL z zapisem ruchów gracza białego. Domyślnie: <code>white_output.jsonl</code> .
--black-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL z zapisem ruchów gracza czarnego. Domyślnie: <code>black_output.jsonl</code> .
--board-width <INT>	(Wymagane) Szerokość planszy do gry (liczba kolumn).
--board-height <INT>	(Wymagane) Wysokość planszy do gry (liczba wierszy).
--seed <INT>	(Opcjonalne) Ziarno dla generatora liczb losowych, kluczowe dla zapewnienia pełnej powtarzalności eksperymentów.
-h, --help	Wyświetla ekran pomocy z opisem wszystkich dostępnych parametrów.
-V, --version	Wyświetla wersję kompilacji programu.

1.2.2 Środowisko turniejowe

Plik wykonywalny `tournament.exe` pozwala na przeprowadzenie rozgrywki pomiędzy dwoma agentami (algorytmami). Interfejs wiersza poleceń zachowuje pełną spójność z graficznym środowiskiem testowym (Rysunek 4).

```
> tournament.exe [OPTIONS] <CONFIG>
```

Rysunek 4: Użycie modułu turniejowego.

Tabela 2: Argumenty i opcje dla programu tournament.

Opcja / Argument	Opis
<CONFIG>	Ścieżka do pliku konfiguracyjnego TOML. Pozycja opcjonalna, domyślnie przyjmuje wartość <code>config.toml</code> .
--white-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL dla gracza białego. Domyślnie: <code>white_output.jsonl</code> .
--black-output <PATH>	(Opcjonalne) Ścieżka do pliku wynikowego JSONL dla gracza czarnego. Domyślnie: <code>black_output.jsonl</code> .
--board-width <INT>	(Wymagane) Szerokość planszy turniejowej.
--board-height <INT>	(Wymagane) Wysokość planszy turniejowej.
--seed <INT>	(Opcjonalne) Ziarno generatora liczb losowych używane do deterministycznej inicjalizacji partii w ramach turnieju.
-h, --help	Wyświetla ekran pomocy programu.
-V, --version	Wyświetla wersję kompilacji programu.

1.3 Format pliku wejściowego

Konfiguracja parametrów agentów biorących udział w rozgrywce odbywa się za pomocą pliku tekstowego w formacie TOML. W pliku tym definiuje się niezależne ustawienia dla obu stron (np. dla gracza białego i czarnego). Każdy z graczy musi mieć jawnie określony typ algorytmu sterującego (pole `type`), co determinuje listę dostępnych parametrów opcjonalnych.

W przypadku pominięcia któregośkolwiek z opcjonalnych pól, środowisko przyjmie predefiniowane wartości domyślne. Przykład struktury takiego pliku przedstawiono na Rysunku 5.

```

board_width = 6
board_height = 7
seed = 100

[white_player]
type = "Minimax"
max_depth = 5
material_weight = 250
advancement_weight = 10

[black_player]
type = "Mcts"
max_iterations = 100000
exploration_constant = 1.41
use_heavy_playouts = true

```

Rysunek 5: Przykładowy plik konfiguracyjny `config.toml`.

W następnych podrozdziałach opisano szczegółowo dostępne parametry dla każdego z trzech zaimplementowanych rodzajów graczy.

1.3.1 Agent Minimax

Wymaga podania parametrów sterujących przeszukiwaniem drzewa gry oraz wag poszczególnych komponentów funkcji heurystycznej oceniającej stan planszy.

Tabela 3: Pola konfiguracyjne dla agenta typu Minimax.

Pole	Domyślnie	Opis
<code>max_depth</code>	4	Maksymalna głębokość przeszukiwania drzewa gry.
<code>material_weight</code>	200	Waga heurystyki odpowiadająca za przewagę materialną (liczbę bierek).
<code>advancement_weight</code>	5	Waga heurystyki premiująca zaawansowanie pionów w kierunku linii docelowej przeciwnika.
<code>defended_weight</code>	5	Waga heurystyki promująca bezpieczne ustawienia, w których piony wzajemnie się bronią.
<code>edge_penalty_weight</code>	-2	Waga (kara) nakładana za umieszczenie pionów na krawędziach planszy.

1.3.2 Agent MCTS

Agent udostępnia on szereg zaawansowanych parametrów pozwalających na włączenie dodatkowych rozszerzeń, takich jak heurystyka RAVE czy Heavy Playouts.

Tabela 4: Pola konfiguracyjne dla agenta typu MCTS.

Pole	Domyślnie	Opis
<code>max_iterations</code>	75000	Maksymalna liczba iteracji algorytmu MCTS (liczba rozbudów drzewa).
<code>max_time_ms</code>	None	Opcjonalny, twardy limit czasu obliczeń dla jednego ruchu w milisekundach. Jeśli jest zdefiniowany, nadpisuje warunek liczby iteracji.
<code>exploration_constant</code>	1.41	Stała eksploracji (C) w formule UCT, balansująca proces poszukiwań.
<code>use_rave</code>	false	Wartość logiczna aktywująca modyfikację RAVE.
<code>rave_k</code>	1000.0	Parametr k sterujący wagą heurystyki RAVE w stosunku do standardowych wartości UCT.
<code>use_heavy_playouts</code>	false	Wartość logiczna włączająca ulepszone symulacje w fazie <i>rollout</i> , kierowane funkcją oceny planszy.
<code>heavy_playouts_epsilon</code>	0.1	Prawdopodobieństwo wykonania ruchu losowego (ϵ) w ulepszonych symulacjach (strategia ϵ -greedy).
<code>material_weight</code>	200	Waga przewagi materialnej, wykorzystywana przez funkcję oceny w fazie <i>heavy playouts</i> .
<code>advancement_weight</code>	5	Waga zaawansowania pionów dla symulacji <i>heavy playouts</i> .
<code>defended_weight</code>	5	Waga bronionych pionów dla symulacji <i>heavy playouts</i> .
<code>edge_penalty_weight</code>	-2	Waga (kara) za piony na krawędzi dla symulacji <i>heavy playouts</i> .

1.3.3 Gracz ludzki

Opcja dedykowana do manualnego testowania środowiska. Pozwala na zdefiniowanie interaktywnego gracza ludzkiego (`type = "Human"`), który wprowadza swoje ruchy poprzez interfejs graficzny GUI. Ten wariant agenta nie wymaga i nie przyjmuje żadnych dodatkowych parametrów konfiguracyjnych.

1.4 Format pliku wyjściowego

Wyniki działania środowiska testowego oraz turniejowego zapisywane są w plikach tekstowych o strukturze JSONL (JSON Lines). Każda linia w takim pliku stanowi niezależny, poprawny obiekt JSON, który agreguje statystyki i parametry z perspektywy pojedynczego gracza w rozegranej partii.

Struktura generowanego logu zawiera zestaw parametrów wspólnych (rejestrowanych dla wszystkich typów graczy, w tym człowieka) oraz sekcję pól specyficznych, które zależą od użytego algorytmu. Przykładowy zrzut danych wyjściowych dla agenta wykorzystującego algorytm Minimax przedstawiono na Rysunku 6.

```

{
  "board_width": 6,
  "board_height": 7,
  "seed": 100,
  "agent_type": "Minimax",
  "agent_color": "Black",
  "agent_won": true,
  "pieces_remaining": 10,
  "move_times_ms": [32, 74, 93, ...],
  "total_moves": 16,
  "moves": ["a6a5", "e6d5", "c6d5", ...],
  "opponent_type": "Mcts",
  "opponent_pieces_remaining": 6,
  "max_depth": 4,
  "total_nodes_evaluated": 33229,
  "total_cutoffs": 10248,
  "material_weight": 200,
  "advancement_weight": 5,
  "defended_weight": 5,
  "edge_penalty_weight": -2
}

```

Rysunek 6: Przykładowy rekord w pliku wyjściowym dla agenta typu Minimax.

W Tabeli 5 zestawiono atrybuty bazowe, które są generowane dla każdego uczestnika rozgrywki (w tym dla agenta typu Human).

Tabela 5: Wspólne pola wyjściowe logowane dla każdego gracza.

Pole	Opis
board_width, board_height	Wymiary planszy, na której rozegrano partię.
seed	Ziarno losowości wykorzystane w danej grze.
agent_type	Typ agenta generującego log (Minimax, Mcts lub Human).
agent_color	Kolor przypisany agentowi w partii (White lub Black).
agent_won	Flaga logiczna informująca, czy dany agent wygrał rozgrywkę.
pieces_remaining	Liczba pionów gracza, które pozostały na planszy w momencie zakończenia gry.
move_times_ms	Lista zawierająca czas obliczania każdego kolejnego ruchu przez agenta (w milisekundach).
total_moves	Całkowita liczba ruchów wykonana przez agenta.
moves	Sekwencja wszystkich wykonanych ruchów (np. "a6a5").
opponent_type	Typ agenta przeciwnika.
opponent_pieces_remaining	Liczba pionów przeciwnika na koniec gry.

W przypadku agentów zautomatyzowanych, bazowy zestaw danych jest rozszerzany o metryki charakterystyczne dla danego modułu heurystycznego. Tabela 6 przedstawia logowane parametry dla algorytmu Minimax, natomiast Tabela 7 dla algorytmu MCTS.

Tabela 6: Pola specyficzne dla agenta Minimax.

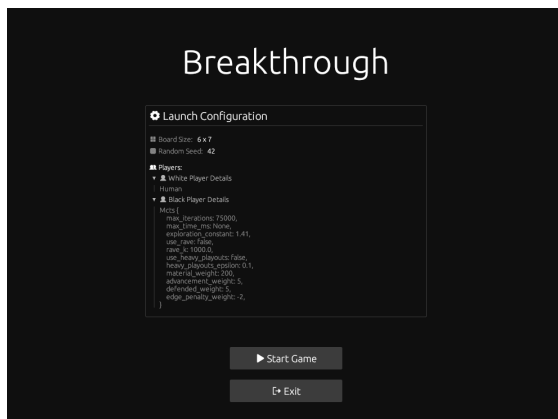
Pole	Opis
max_depth	Skonfigurowana maksymalna głębokość przeszukiwania.
total_nodes_evaluated	Łączna liczba wywołań funkcji statycznej oceny stanu gry (odwiedzonych liści) w trakcie całej partii.
total_cutoffs	Łączna liczba odcięć uzyskanych w całym drzewie przeszukiwań.
material_weight ...	Zestaw wag wykorzystanych w funkcji oceny (material, advancement, defended, edge_penalty).

Tabela 7: Pola specyficzne dla agenta MCTS.

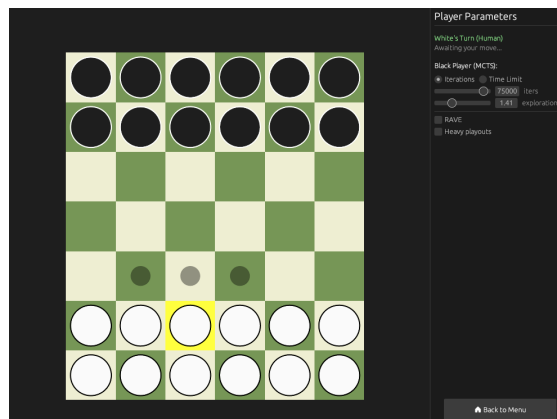
Pole	Opis
<code>max_iterations</code>	Limit liczby iteracji algorytmu MCTS.
<code>max_time_ms</code>	Limit czasu obliczeń dla jednego ruchu (lub <code>null</code> , jeśli brak).
<code>exploration_constant</code>	Wartość współczynnika eksploracji.
<code>use_rave, rave_k</code>	Informacja o wykorzystaniu heurystyki RAVE i jej parametrze k .
<code>use_heavy_playouts ...</code>	Flaga logiczna i parametry (<code>epsilon</code> oraz wagi heurystyki) wykorzystywane do kierowania fazą symulacji (rollout).
<code>total_iterations</code>	Zsumowana liczba wszystkich iteracji MCTS wykonanych przez agenta we wszystkich ruchach danej partii.
<code>total_nodes_created</code>	Sumaryczna liczba wygenerowanych w pamięci węzłów drzewa MCTS podczas gry.

1.5 Interfejs graficzny

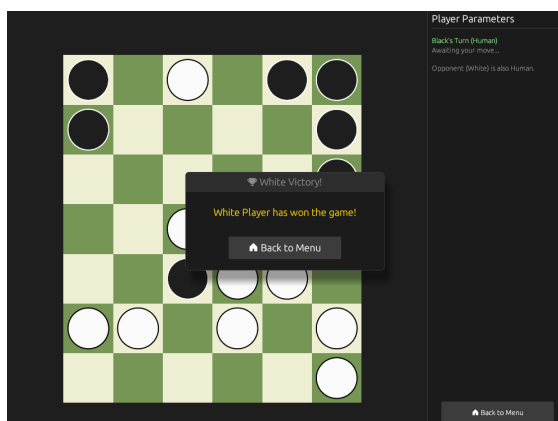
Aplikacja została wyposażona w prosty, czytelny interfejs graficzny (GUI), umożliwiający rozgrywkę człowieka z agentem. Zestawienie poszczególnych widoków aplikacji przedstawiono na Rysunku 7.



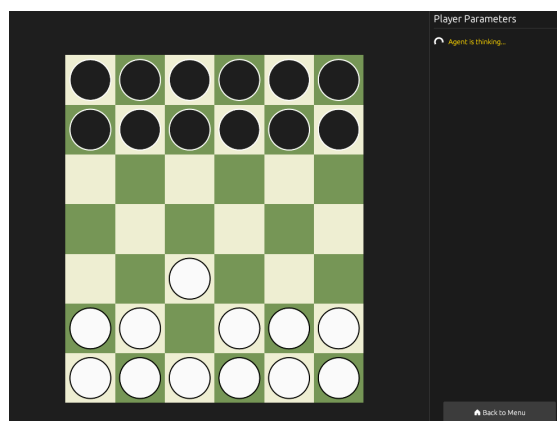
(a) Ekran startowy.



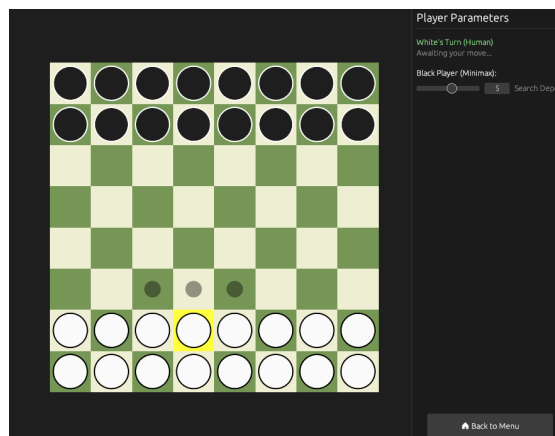
(b) Oczekiwanie na ruch gracza.



(c) Ekran końcowy z ogłoszeniem zwycięzcy.



(d) Oczekiwanie na ruch agenta.



(e) Gra z agentem Minimax.

Rysunek 7: Zrzuty ekranu interfejsu graficznego dla gry Breakthrough.

2 Weryfikacja hipotez badawczych

2.1 Hipoteza 1: Wpływ algorytmu RAVE

2.2 Hipoteza 2: Wpływ algorytmu Heavy Playouts

2.3 Hipoteza 3: Przewaga nad ewaluacją statyczną

2.3.1 Metodologia i przebieg eksperymentu

Weryfikacja hipotezy wymagała przeprowadzenia serii rozgrywek turniejowych, w których zautomatyzowani agenci bazujący na algorytmie MCTS zostali skonfrontowani z agentem heurystycznym, wykorzystującym algorytm Minimax ze statyczną funkcją oceny pozycji. Celem eksperymentu było zbadanie, czy probabilistyczne podejście do przeszukiwania przestrzeni stanów pozwala na uzyskanie przewagi nad deterministycznym oponentem o ograniczonym horyzoncie planowania.

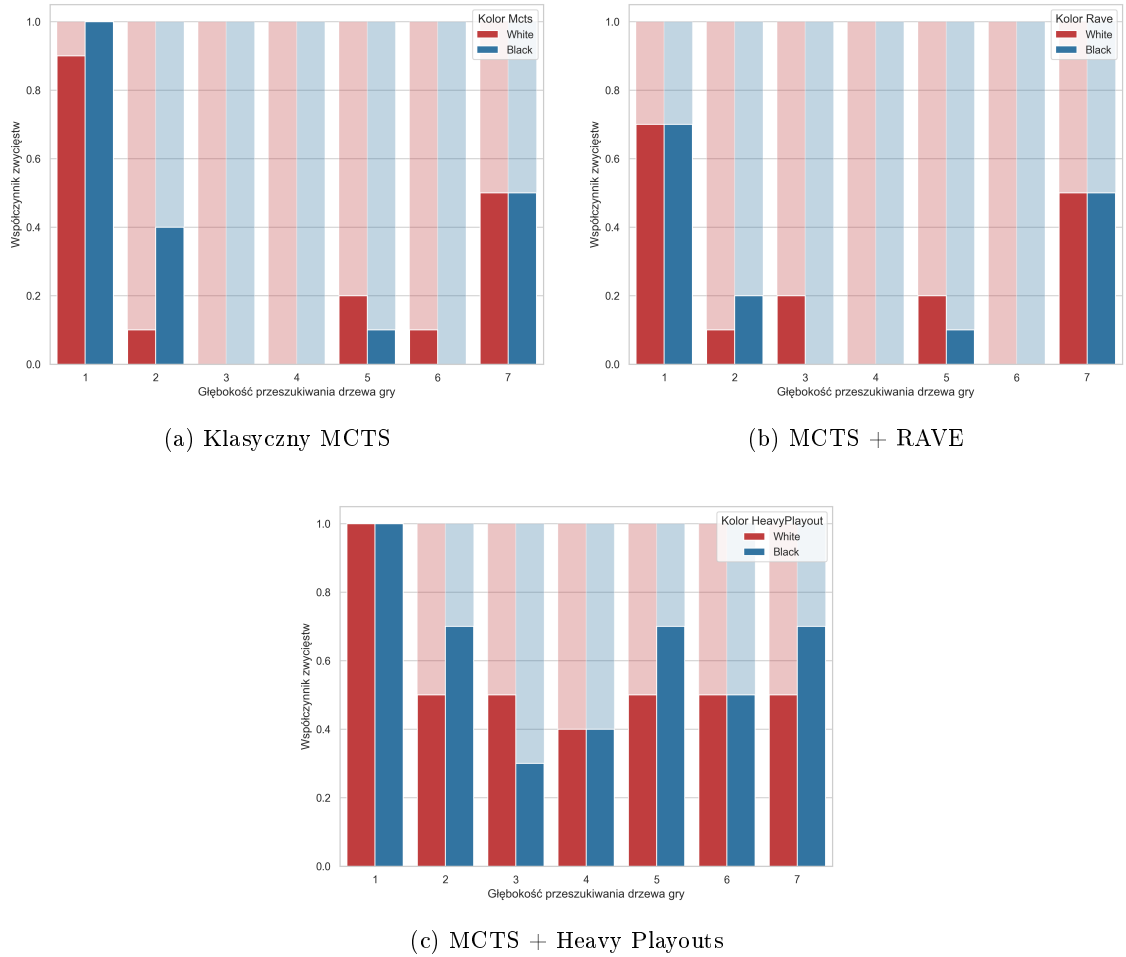
Eksperyment podzielono na trzy główne serie pomiarowe, odpowiadające testowanym wariantom algorytmu sterującego: MCTS, MCTS + RAVE oraz MCTS + Heavy Playouts. W każdej z serii agent testowy (korzystający ze stałego budżetu iteracyjnego 10000) rozgrywał partie przeciwko agentowi Minimax. Siła oponenta była stopniowo zwiększana poprzez inkrementację limitu maksymalnej głębokości przeszukiwania drzewa gry (parametr d) w przedziale od $d = 1$ do $d = 7$. Pozwoliło to na weryfikację zachowania wariantów stochastycznych w starciu z przeciwnikiem o rosnących zdolnościach taktycznych.

Dla każdej testowanej głębokości oponenta rozegrano 20 niezależnych partii. Aby zneutralizować wpływ przewagi pierwszego ruchu, zrezygnowano z przypisywania stałych ról — w 10 partiach agent kontrolował piony białe, a w pozostałych 10 piony czarne. Tym samym każda z trzech serii pomiarowych składała się ze 140 partii, co przełożyło się na 420 rozegranych gier w całym eksperymencie.

Zgodnie z przyjętymi założeniami, główną metryką decydującą o weryfikacji hipotezy H3 był sumaryczny odsetek wygranych partii (wymagany próg powyżej 70%). Dodatkowo, w celu pogłębienia analizy, z zebranych logów wyodrębniono informacje na temat średniej długości trwania rozgrywki (liczby ruchów) oraz czasów namysłu poszczególnych algorytmów.

2.3.2 Analiza wyników

Zestawienie odsetka wygranych partii dla wszystkich trzech wariantów algorytmu MCTS (MCTS, MCTS + RAVE oraz MCTS + Heavy Playouts) w zależności od głębokości przeszukiwania agenta Minimax przedstawiono na Rysunku 8 oraz w Tabeli 8.



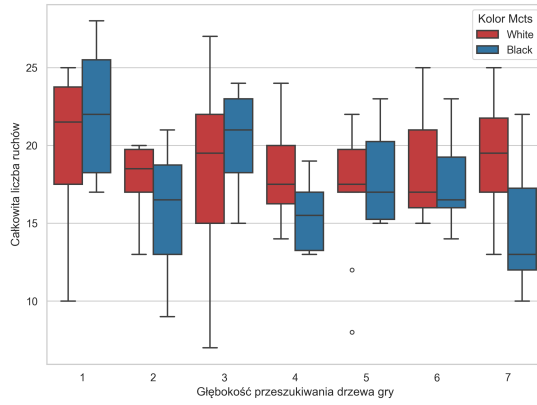
Rysunek 8: Porównanie odsetka wygranych partii w zależności od głębokości przeszukiwania agenta Minimax dla różnych wariantów algorytmu MCTS.

Bezpośrednie porównanie tych wykresów pozwala na sformułowanie kilku obserwacji dotyczących stabilności i siły gry poszczególnych modyfikacji:

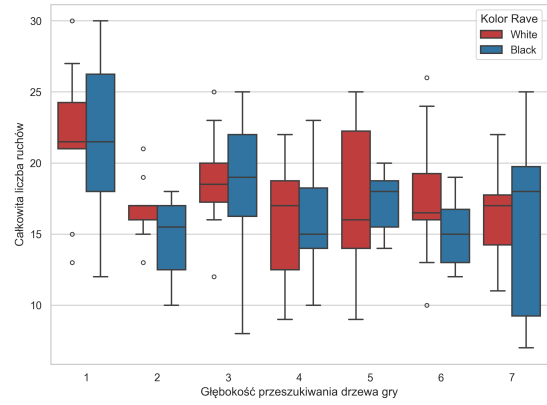
1. Niezależnie od zastosowanego wariantu MCTS, można zaobserwować wyraźny trend spadku skuteczności w początkowej fazie wykresu. Dla najmniejszej głębokości ($d = 1$), wszystkie modyfikacje oparte na symulacjach całkowicie dominują agenta Minimax, osiągając wskaźniki zwycięstw bliskie lub równe 100% ($\geq 70\%$). Jednakże, wraz ze wzrostem limitu głębokości do $d = 3$ i $d = 4$, siła gry Minimaxa znacząco rośnie, co przekłada się na gwałtowny spadek skuteczności agentów probabilistycznych. W tym przedziale klasyczny MCTS oraz MCTS z modyfikacją RAVE przegrywają niemal wszystkie partie, co obnaża podatność losowych oraz pół-losowych symulacji na szerszy horyzont planowania agenta Minimax.
2. Szczególnie interesujące jest zachowanie algorytmów dla dużych głębokości przeszukiwania oponenta ($d \geq 5$). Na wykresach zauważalne jest swoiste odwrócenie trendu — skuteczność wariantów MCTS ponownie zaczyna rosnąć, a dla $d = 7$ agent Minimax zaczyna regularnie przegrywać z ulepszonymi wariantami (Heavy Playouts i RAVE osiągają lepsze wyniki w tym rejonie w porównaniu do klasycznego MCTS, choć i on odnotowuje wzrost do poziomu około 50%). Przy bardzo dużych głębokościach (np. $d = 7$), agent Minimax staje się ofiarą własnego determinizmu — „ślepo” ufa zaprogramowanym wagom heurystycznym. Warianty MCTS, dzięki pełnym symulacjom do końca partii, potrafią elastyczniej identyfikować rzeczywiste szanse na wygraną.
3. Spośród testowanych modyfikacji, wariant MCTS + Heavy Playouts (Rysunek 8c) wykazał się największą odpornością na siłę taktyczną Minimaxa w środkowym przedziale głębokości ($d = 3$, $d = 4$).
4. RAVE (Rysunek 8b) zachowywało się w sposób zbliżony do klasycznego MCTS.

Tabela 8: Odsetek wygranych partii przez różne warianty algorytmu MCTS w starciu z algorytmami Minimax o zmiennej głębokości przeszukiwania (d).

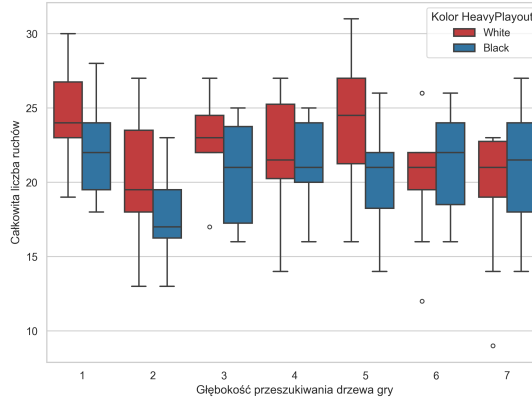
Głębokość	MCTS	MCTS + RAVE	MCTS + Heavy Playouts
1	95.0%	70.0%	100.0%
2	25.0%	15.0%	60.0%
3	0.0%	10.0%	40.0%
4	0.0%	0.0%	40.0%
5	15.0%	15.0%	60.0%
6	5.0%	0.0%	50.0%
7	50.0%	50.0%	60.0%



(a) Klasyczny MCTS



(b) MCTS + RAVE



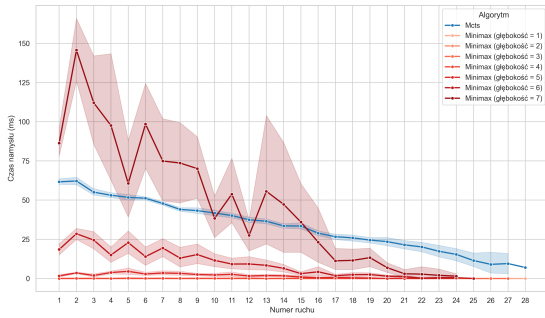
(c) MCTS + Heavy Playouts

Rysunek 9: Zestawienie dystrybucji całkowitej liczby ruchów rozegranych w partiach przeciwko agentowi Minimax dla różnych wariantów algorytmu MCTS.

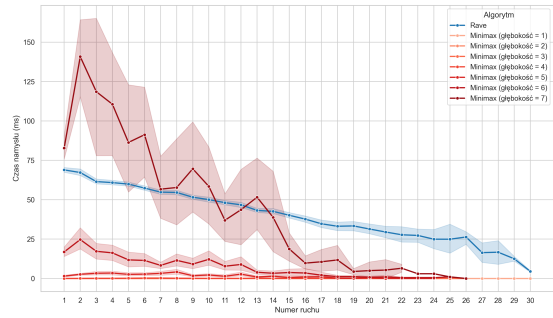
Kolejnym elementem poddanym analizie była średnia długość rozegranych partii, wyrażona jako całkowita liczba ruchów wykonana przez obu graczy przed osiągnięciem stanu końcowego (Rysunek 9, Tabela 9). Wykresy uwzględniają odrębne „pudełka” dla sytuacji, w której agent MCTS steruje bierkami białymi oraz czarnymi.

Tabela 9: Średnia długość partii (wyrażona jako całkowita liczba ruchów obu graczy) wraz z odchyleniem standardowym, w starciach wariantów MCTS z algorytmem Minimax.

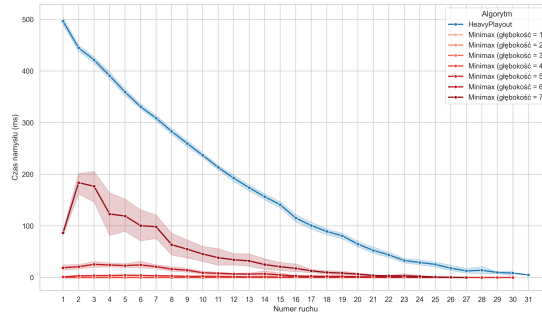
Głębokość	MCTS	MCTS + RAVE	MCTS + Heavy Playouts
1	21.1 $\pm$ 4.3	21.5 $\pm$ 5.5	23.4 $\pm$ 3.5
2	17.0 $\pm$ 3.2	15.8 $\pm$ 2.6	19.1 $\pm$ 3.9
3	19.6 $\pm$ 4.7	18.6 $\pm$ 4.5	21.8 $\pm$ 3.4
4	16.9 $\pm$ 2.9	16.1 $\pm$ 4.1	21.8 $\pm$ 3.3
5	17.4 $\pm$ 3.5	17.4 $\pm$ 3.9	22.1 $\pm$ 4.2
6	17.9 $\pm$ 3.1	16.3 $\pm$ 3.9	21.0 $\pm$ 3.8
7	17.1 $\pm$ 4.7	15.8 $\pm$ 4.9	20.2 $\pm$ 4.4



(a) Klasyczny MCTS



(b) MCTS + RAVE



(c) MCTS + Heavy Playouts

Rysunek 10: Średni czas namysłu (w milisekundach) dla różnych wariantów algorytmu MCTS oraz ich przeciwników (agenta Minimax o rosnącej głębokości przeszukiwania).

Ostatnim, w ramach tego eksperymentu, badanym elementem była analiza średniego czasu namysłu wymaganego do wygenerowania pojedynczego ruchu. Profile wydajnościowe, zestawione na układzie wykresów (Rysunek 10), obrazują różnice w naturze obu podejść do przeszukiwania przestrzeni stanów.

Przewidywalną obserwacją, jest wykładniczy wzrost czasu namysłu algorytmu Minimax wraz ze wzrostem limitu głębokości przeszukiwania (d). Dla płytkich wartości ($d \leq 4$), decyzje podejmowane są niemal natychmiastowo, jednakże przekroczenie progu $d = 5$ powoduje drastyczny wzrost wielkości drzewa gry (wynikające z wysokiego współczynnika rozgałęzienia w gry). Dla maksymalnej testowanej głębokości ($d = 7$), pojedynczy ruch oponenta statycznego zajmuje znacząco więcej czasu niż iteracyjne rozwijanie drzewa przez warianty MCTS oraz MCTS + RAVE, co pokazuje ograniczenia skalowalności naiwnego (bądź wyposażonego jedynie w cięcia alfa-beta) przeglądu zupełnego.

Ciekawszym zjawiskiem jest zachowanie czasu namysłu dla algorytmów opartych na MCTS. Mimo iż operowały one na stałym, odgórnie narzuconym limicie całkowitej liczby iteracji w każdym ruchu (niezależnie od przeciwnika), czas potrzebny na realizację tego budżetu skraca się (w przybliżeniu liniowo) wraz z rozwojem gry. Losowe lub pół-losowe rozgrywki algorytmów MCTS trwają coraz krócej ponieważ pozostało niewiele ruchów do zakończenia rozgrywki.

Analizując warianty MCTS między sobą, można zauważyć, że zastosowanie heurystyki RAVE nie wprowadziło znaczącego narzutu obliczeniowego, natomiast wielokrotne wywoływanie funkcji oceny pozycji w wariantcie Heavy Playouts, dosyć znacząco wydłuża czas namysłu algorytmu, którego średni czas namysłu do 11 ruchu przekracza 200 ms (przy stałym limicie 10000 iteracji).

2.3.3 Wnioski końcowe

Analiza zebranych danych pozwala stwierdzić, że dla najbardziej podstawowego wariantu płytkiego przeszukiwania, czyli dla głębokości $d = 1$, wszystkie testowane modyfikacje algorytmu stochastycznego z sukcesem osiągnęły lub przekroczyły wymagany próg skuteczności. Klasyczny wariant MCTS uzyskał wynik 95.0%, MCTS z heurystyką RAVE osiągnął barierę 70.0%, natomiast wariant kierowany (Heavy Playouts) zdominował przeciwnika całkowicie, wygrywając 100.0% rozegranych partii.

Hipoteza H3 zostaje przyjęta. Należy jednak zaznaczyć, że założony próg 70% wygranych jest spełniony wyłącznie dla głębokości równej 1. Zwiększenie horyzontu planowania agenta Minimax zaledwie o jeden lub dwa półruchy ($d \geq 2$) powodowało gwałtowne załamanie skuteczności metod opartych o algorytm MCTS.

2.4 Hipoteza 4: Wpływ rozmiaru planszy na asymetrię szans

2.4.1 Metodologia i przebieg eksperymentu

Weryfikacja czwartej hipotezy badawczej (H4) koncentrowała się na ocenie wpływu geometrii planszy na asymetrię szans w grze. Ze względu na specyfikę mechaniki Breakthrough, w której celem jest dotarcie do linii końcowej przeciwnika, gracz wykonujący pierwszy ruch (białe) dysponuje naturalną przewagą tempa. Hipoteza zakładała, że wraz ze wzrostem wymiarów planszy przewaga koloru białego ulegnie marginalizacji na rzecz strategii. Zgodnie z hipotezą, zmodyfikowane algorytmy MCTS będą w stanie odnosić relatywnie częstsze zwycięstwa z pozycji czarnych na większych planszach.

Rozgrywki przeprowadzono dla asymetrycznych wymiarów planszy. Szerokość ustalono jako stałą wartość $W = 8$, natomiast wysokość (H) stanowiła zmienną niezależną przyjmującą wartości z rosnącego zbioru: $H \in \{5, 6, 7, 8, 9, 10, 11\}$.

Eksperyment podzielono na trzy symetryczne serie pomiarowe, w których po obu stronach planszy stawali identyczni agenci:

1. **Seria 1:** Klasyczny algorytm MCTS (UCT) grający przeciwko sobie.
2. **Seria 2:** Wariant MCTS + RAVE.
3. **Seria 3:** Wariant MCTS + Heavy Playouts.

Aby zapewnić agentom budżet adekwatny do rozmiaru gry, maksymalna liczba iteracji zmieniała się liniowo w zależności od wysokości planszy zgodnie ze wzorem:

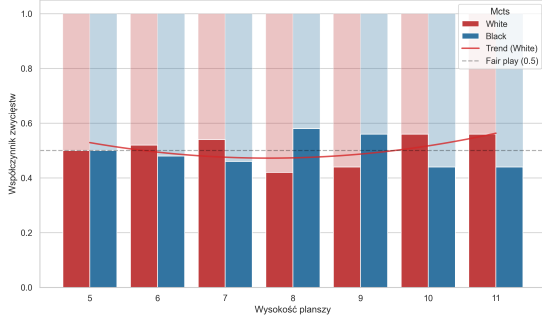
$$I = 2000 \cdot H$$

Tym samym dla planszy 8×5 limit wynosił 10000 iteracji, a dla 8×11 już 22000.

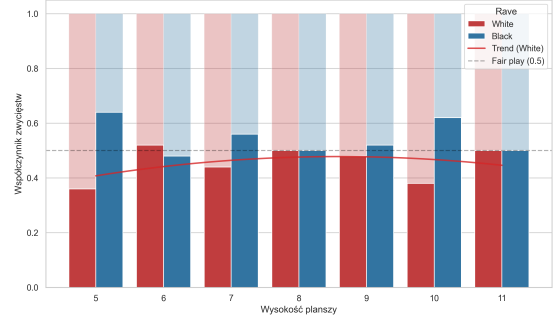
Dla każdego z testowanych wymiarów planszy i dla każdej z trzech serii rozegrano dokładnie 50 partii turniejowych. Ponieważ w każdej serii agenci reprezentowali identyczną siłę gry, zrezygnowano z rotacji kolorów na rzecz jednorodnego monitorowania współczynnika zwycięstw z perspektywy białego gracza.

2.4.2 Analiza wyników

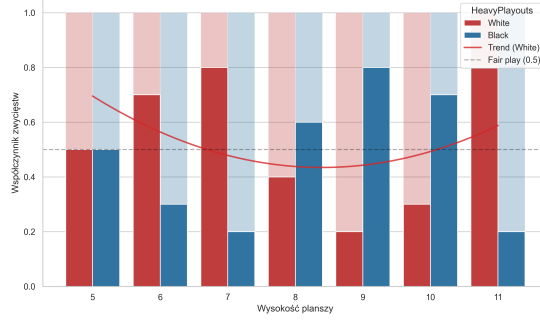
Zgromadzone dane pozwoliły na prześledzenie zachowania wskaźnika zwycięstw dla gracza rozpoczynającego (Białe) w zależności od wysokości planszy H . Zestawienie wyników dla wszystkich trzech symetrycznych par algorytmów (Klasyczny MCTS, MCTS + RAVE, MCTS + Heavy Playouts) przedstawiono w formie wykresów na Rysunku 11. Szczegółowe dane liczbowe zostały zgromadzone w Tabeli 10.



(a) Klasyczny MCTS



(b) MCTS + RAVE



(c) MCTS + Heavy Playouts

Rysunek 11: Odsetek zwycięstw gracza rozpoczynającego (Białe) w starciach symetrycznych w zależności od wysokości planszy (H) przy stałej szerokości $W = 8$.

Wstępna analiza wyników wskazuje na dużą zmienność (wariancję) wskaźnika wygranych w poszczególnych wariantach na szerokim przedziale wysokości planszy. W celu ułatwienia interpretacji tych odchyleń, na wykresach naniesiono linię trendu obrazującą ogólną tendencję zmian skuteczności (modelowaną przez wielomian stopnia 2), a także poziomą linię wyznaczającą próg 50% (tzw. linię *fair play*), która reprezentuje stan idealnej równowagi szans między stroną rozpoczynającą a broniącą.

Analiza zgromadzonych danych pozwala na pozytywne zweryfikowanie postawionych warunków. Rozpatrując kluczowy przedział wysokości od $H = 7$ do $H = 9$, zaobserwowano spadek odsetka wygranych Białego. Dla klasycznego MCTS skuteczność Białych spadła z poziomu 54.0% ($H = 7$) do 42.0% ($H = 8$) oraz 44.0% ($H = 9$), co oznacza utratę inicjatywy startowej i przejście inicjatywy (choć z niewielką przewagą) na stronę Czarnych.

Wprowadzenie wiedzy domenowej poprzez modyfikację Heavy Playouts ujawniło wyraźny trend malejący: od dominacji Białych wynoszącej 80.0% na planszy 8×7 , poprzez spadek do 40.0% na planszy 8×8 , aż do zaledwie 20.0% na planszy 8×9 .

Należy zaznaczyć, że trend ten nie zachowuje idealnej liniowości w nieskończoność (co pokazują anomalie dla dużej planszy $H = 11$), a wariant RAVE charakteryzował się większą zachowawczością wyników wokół linii remisu (50%). Nie zmienia to jednak faktu, że w zakładanym w kryteriach przedziale badawczym (odpowiednik plansz średnich i dużych) zjawisko marginalizacji przewagi Białych jest zauważalne.

Tabela 10: Zestawienie skuteczności i średniej długości partii dla wariantów MCTS w zależności od wysokości planszy.

Wariant	Wysokość planszy	Współczynnik zwycięstw (Białe/Czarne)	Średnia długość partii
Klasyczny MCTS	5	50/50%	15.8 ± 3.5
	6	52/48%	20.4 ± 4.1
	7	54/46%	25.4 ± 6.2
	8	42/58%	29.1 ± 7.7
	9	44/56%	31.3 ± 7.2
	10	56/44%	39.2 ± 8.4
	11	56/44%	38.5 ± 10.7
MCTS + Heavy Playouts	5	50/50%	16.3 ± 1.7
	6	70/30%	19.8 ± 4.7
	7	80/20%	27.5 ± 2.6
	8	40/60%	34.9 ± 5.1
	9	20/80%	40.7 ± 4.0
	10	30/70%	52.7 ± 6.4
	11	80/20%	56.5 ± 3.8
MCTS + RAVE	5	36/64%	15.2 ± 3.2
	6	52/48%	18.4 ± 4.1
	7	44/56%	19.7 ± 4.3
	8	50/50%	23.0 ± 5.2
	9	48/52%	25.6 ± 5.9
	10	38/62%	28.6 ± 7.0
	11	50/50%	30.9 ± 8.5

2.4.3 Wnioski końcowe

Hipoteza 4 zostaje przyjęta. Rozmiar planszy w grze Breakthrough w istotne znaczenie na balans rozgrywki, a na rozszerzonych planszach przemyślana strategia (reprezentowana przez ukierunkowane symulacje) pozwala na statystycznie istotne przejście kontroli nad partią przez gracza broniącego (Czarne).

3 Testy z udziałem ludzi

4 Podsumowanie

Literatura

- [1] *The Rust Programming Language – Installation*, <https://doc.rust-lang.org/book/ch01-01-installation.html#installation> (dostęp 28.01.2026r.).
- [2] *What is rustc?*, <https://doc.rust-lang.org/stable/rustc/index.html#what-is-rustc> (dostęp 28.01.2026r.).
- [3] *The Cargo Book – Installation*, <https://cargo-book.rust-lang.org/getting-started/installation.html> (dostęp 28.01.2026r.).